

Track A→ RBC Project: “Mini Banking Backend”

Scenario

You are building a backend service that supports basic banking operations, including account management, money transfers, and transaction history tracking. The system must expose REST APIs that allow clients to retrieve account information, perform transfers between accounts, view transaction history and access basic banking analytics.

The database required for this assignment is provided to you on Brightspace under the folder:

Week3/Track_B_RBC/RBC_Db.sql

Figure 1 shows the EER Diagram of the database we are using in this assignment. Make sure that you import and run the SQL script successfully before starting your implementation.

General Instructions

Make sure that you are following the general instructions of:

- Using FastAPI as your backend framework
- Using SQLAlchemy ORM for database interaction
- Configuring connection pooling in your engine
- Using a .env file for database credentials
- Testing your endpoints using Postman
- Verifying your API documentation in Swagger/OpenAPI
- Your application must run successfully using:

Command prompt

uvicorn main:app --reload

Include a README.md file explaining:

- How to install dependencies
- How to configure the database
- How to run the application

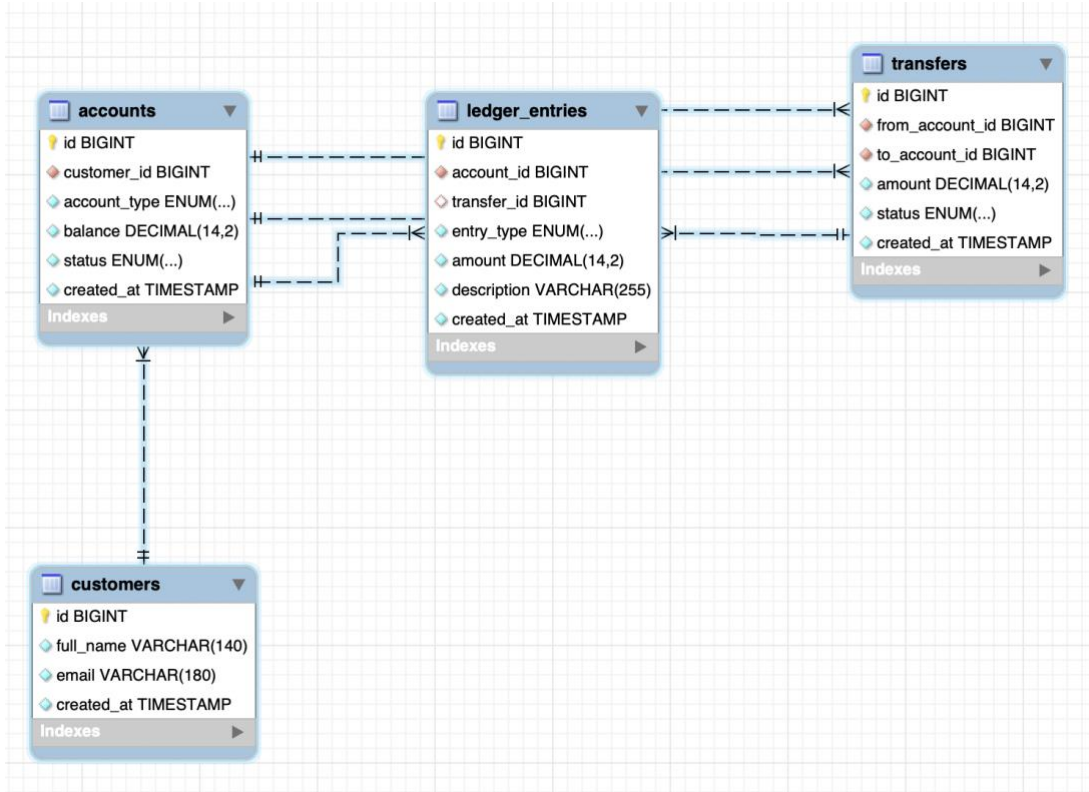


Figure 1.RBC Database EER Diagram

Part 1 – Account APIs

1. GET /accounts

What is expected:

You must build a filtering endpoint that allows retrieval of accounts based on account type and status.

This endpoint must:

- Support optional query parameters:
 - account_type (checking, savings)
 - status (active, frozen)

Example:

GET /accounts?account_type=checking&status=active

Your implementation should dynamically apply filters if parameters are provided.

2. Offset-Based Pagination

Endpoint:

GET /accounts/paged

What is expected:

Implement offset-based pagination using:

- limit
- offset

Example:

GET /accounts/paged?limit=5&offset=10

This should return only a subset of accounts.

3. Cursor-Based Pagination

Endpoint:

GET /accounts/cursor

What is expected:

Implement cursor-based pagination using stable ordering based on:

- created_at
- then id

The endpoint must:

- Accept limit
- Accept cursor
- Return the next page after the cursor

In your README, briefly explain why cursor-based pagination is more scalable than offset-based pagination.

Part 2 – Transfer API

4. POST /transfers

What is expected:

You must implement full transfer logic between two accounts.

Input:

- from_account_id
- to_account_id
- amount

The endpoint must:

1. Validate both accounts exist
2. Validate that both accounts are active
3. Validate that amount > 0
4. Check that the sender has sufficient balance
5. Subtract amount from sender
6. Add amount to receiver
7. Create transfer record
8. Create corresponding ledger entries (debit and credit)

Important:

Do NOT implement explicit transaction management yet. This is intentional and will be addressed in Week 4.

5. GET /transfers/{transfer_id}

What is expected:

Return complete transfer details including:

- Transfer record
- Associated ledger entries
- Basic account information

This demonstrates proper relationship handling using ORM joins.

6. GET /accounts/{account_id}/ledger

What is expected:

Return the transaction history (ledger entries) for a specific account.

You must:

- Order results by created_at descending
- Support pagination (limit, offset)

This endpoint simulates real-world banking transaction history APIs.

Part 3 – Analytics (ORM vs Raw SQL)

Create two endpoints:

- /analytics/high-activity-customers/orm
- /analytics/high-activity-customers/sql

What is expected:

Return the top 5 customers ranked by number of ledger entries in the last 30 days.

Each response must include:

- customer_id
- full_name
- transaction_count

One implementation must use SQLAlchemy ORM aggregation.

The other must use raw SQL with GROUP BY and COUNT.

Reflection (Required)

Write a short technical reflection (maximum 200 words) the following question in the **Part3_Answers.txt** file in your git repository:

- Which implementation was easier?
- Which gives more control?
- When would you prefer raw SQL in a banking system?

Focus on trade-offs, not just personal preference.

Part 4 – Connection Pooling

What is expected:

Configure your SQLAlchemy engine with:

- `pool_size`
- `max_overflow`
- `pool_pre_ping=True`

In your **Part4_Answers.txt**, answer:

1. What happens if 100 users attempt transfers simultaneously?
2. What happens if `pool_size = 3`?
3. Why is connection pooling especially important in banking systems?

What I will look for

- Clean filtering queries
- Correct pagination behavior
- Correct join and aggregation in analytics